

COMSM1302 Lab Sheet 2 (Solutions)

Half adder

The truth table for C_{out} is the exact same as for $A \wedge B$ - for implementation see subcircuit <i>half adder</i> .	A	B	C_{out}	The truth table for S is the exact same as for $A \oplus B$ - for implementation see subcircuit <i>half adder</i> .	A	B	S
	0	0	0		0	0	0
	0	1	0		0	1	1
	1	0	0		1	0	1
	1	1	1		1	1	0

An incrementer adds 1 to its input. Since the half adder subcircuit only accepts 1-bit signals, we first split the 4-bit input into four separate bits. Each bit is processed by a half adder, so we need four copies of the subcircuit. There is no B input, so this pin on each adder is connected to the C_{out} of the previous one, except for the LSB adder, where B is set to a constant 1. Finally, we recombine the four S outputs into a single 4-bit signal with a splitter. For implementation see subcircuit *4-bit incrementer*.

Full adder

C_{in} AB	0	1
00	0	0
01	0	1
11	1	1
10	0	1

$$C_{out} \equiv (A \wedge B) \vee (C_{in} \wedge (A \oplus B))$$

S AB	0	1
00	0	1
01	1	0
11	0	1
10	1	0

$$S \equiv A \oplus B \oplus C$$

Our logic for the C_{out} and S outputs on the full adder can be found through the above Karnaugh maps, which is simplified to the formula in red. We can replace $A \vee B$ with $A \oplus B$ in the C_{out} formula because they differ only when $A = B = 1$. In that case, $(A \wedge B) = 1$, so the whole expression is already 1 regardless of the second term. For further details, see [lecture 2-2: binary addition](#) and for implementation see subcircuit *full adder*.

A full adder can also be built from two half adders: HA1 computes $A + B$, and HA2 computes $(A + B) + C_{in}$. This produces two carry signals, which are combined with OR to give a single C_{out} . A truth table confirms this:

HA1 C_{out}	HA2 C_{out}	FA C_{out}	
0	0	0	e.g. when $A = 0, B = 0, C_{in} = 0$
0	1	1	e.g. when $A = 0, B = 1, C_{in} = 1$
1	0	1	e.g. when $A = 1, B = 1, C_{in} = 0$
1	1	X	we never gets these at the same time

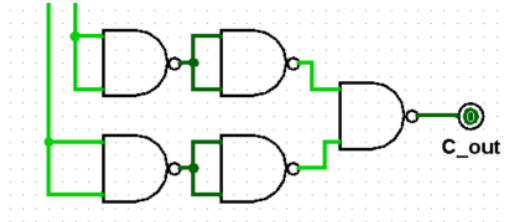
The 4-bit adder can then be constructed using 4 copies of the full adder subcircuit, as explained [at the end of the binary addition lecture](#) i.e. connecting each C_{out} to the next C_{in} . Splitters divide A and B into 1-bit inputs and recombine the sum bits into the 4-bit output. For implementation see subcircuit *4-bit adder*.

To extend this design to subtraction, we input either B (for addition) or $-B$ i.e. $\neg B + 1$ (for subtraction). A multiplexer selects between B and $\neg B$. The unused C_{in} of the LSB adder is then connected to the multiplexer's control signal, ensuring that in subtraction mode it adds the extra 1. Thus, when control = 1, the circuit performs $A - B$; when control = 0, it performs $A + B$. For implementation see subcircuit *4-bit adder/subtractor*.

NAND adders

From the previous lab, we know an XOR can be built with 4 NAND gates (though being able to prove this is non-examinable, and using the 5 NAND gate configuration would be sufficient), which we can use to generate S . For C_out , which requires AND, we need 2 NAND gates. Since the first NAND in the XOR already computes $\neg(A \wedge B)$, we can reuse that signal and pass it through one more NAND to obtain C_out . For implementation see [subcircuit nand half adder](#).

To extend this to a full adder, we duplicate the half adder design and connect the copies like in the standard version. The only addition needed is the NAND equivalent of an OR gate. Adding this produces a 5-NAND configuration, which simplifies to a single NAND by removing double negations:



For implementation see [subcircuit nand full adder](#).

Plexers

Sel \ AB	0	1
00	0	0
01	0	1
11	1	1
10	1	0

For implementation of 2-to-1 multiplexer, based on this Karnaugh map, see [subcircuit 2-1 mux](#).

The truth table for $Out0$ is the same as for $In \wedge \neg Sel$, while the truth table for $Out1$ is the same as for $In \wedge Sel$ - for implementation see [subcircuit 1-2 demux](#).

In	Sel	Out0	In	Sel	Out1
0	0	0	0	0	0
0	1	0	0	1	0
1	0	1	1	0	0
1	1	0	1	1	1

$$Out \equiv (A \wedge \neg Sel) \vee (B \wedge Sel)$$

A 4-to-1 multiplexer selects one of four inputs using two select bits. Each select bit essentially decides *between two options*. Since a 2-to-1 multiplexer already performs a single binary choice, we can combine them hierarchically:

- The first select bit chooses *within each pair* of inputs (e.g. $In0$ vs $In1$, $In2$ vs $In3$), which needs two 2-to-1 multiplexers.
- The second select bit then chooses *between those two results*, which requires a third 2-to-1 multiplexer.

So the structure directly reflects how the two select bits encode the choice out of four: one bit narrows it down to a pair, the other decides within the pair. For implementation see [subcircuit 4-1 mux](#).

A 1-to-4 demultiplexer works in the reverse way. One input needs to be directed to one of four outputs, according to two select bits. A 1-to-2 demux handles a single binary decision, so we cascade them:

- The first select bit decides which *pair of outputs* will receive the signal.
- The second select bit then decides *which output within that pair*.

Again, the hierarchy mirrors the binary structure of the select lines: two bits $\rightarrow$ two stages of binary decision. For implementation see [subcircuit 1-4 demux](#).

Arithmetic Logic Unit

The Hack ALU can be built using 2-1 multiplexers. Order matters - zeroing must happen before possible negation, the function f must be applied to the post-processed operands, and finally the no negates the resulting signal. If you forget the order, reconstruct it from the Hack truth table!

The zr output is true if (and only if) all the outputs bits are 0 i.e. $\neg bit0 \wedge \neg bit1 \wedge \neg bit2 \wedge \neg bit3 \equiv \neg(bit0 \vee bit1 \vee bit2 \vee bit3)$, while the ng flag is true if (and only if) the output's MSB is 1 i.e. $bit3$. For implementation see [subcircuit alu](#).

Number representations

Binary	Octal	Decimal (2's complement)	Hexadecimal	Decimal (signed magnitude)	Decimal (1's complement)	Floating point
10101101 ₂	655 ₈	-83 ₁₀	AD ₁₆	-45 ₁₀	-82 ₁₀	-1.40625 ₁₀
01110101 ₂	165 ₈	117 ₁₀	75 ₁₆	117 ₁₀	117 ₁₀	6.625 ₁₀
11011100 ₂	734 ₈	-36 ₁₀	DC ₁₆	-92 ₁₀	-35 ₁₀	-3.75 ₁₀
10000101 ₂	605 ₈	-123 ₁₀	85 ₁₆	-5 ₁₀	-122 ₁₀	-0.578125 ₁₀
11001010 ₂	712 ₈	-54 ₁₀	CA ₁₆	-74 ₁₀	-53 ₁₀	-2.625 ₁₀
00101001 ₂	051 ₈	41 ₁₀	29 ₁₆	41 ₁₀	41 ₁₀	1.28125 ₁₀
01111011 ₂	173 ₈	123 ₁₀	7B ₁₆	123 ₁₀	123 ₁₀	7.375 ₁₀
11111101 ₂	775 ₈	-3 ₁₀	FD ₁₆	-125 ₁₀	-2 ₁₀	-7.625 ₁₀
10101100 ₂	654 ₈	-84 ₁₀	AC ₁₆	-44 ₁₀	-83 ₁₀	-1.375 ₁₀
10010010 ₂	622 ₈	-110 ₁₀	92 ₁₆	-18 ₁₀	-109 ₁₀	-0.78125 ₁₀
01111111 ₂	177 ₈	127 ₁₀	7F ₁₆	127 ₁₀	127 ₁₀	7.875 ₁₀
11011010 ₂	732 ₈	-38 ₁₀	DA ₁₆	-90 ₁₀	-37 ₁₀	-3.625 ₁₀

For floating point, recall the format here is S(1), E(2), M(5). The decoded value depends on these bit widths - changing the number of exponent or mantissa bits would change the result.

For signed integers, note the pattern: 2's complement, 1's complement, and signed magnitude all give the same decimal values for positive numbers - they only differ in how negatives are represented.